

Tugas Proyek Kelompok

Sistem Operasi

Febriyansyah Ramadhan, S.Kom., M.T.

Tugas Proyek Kelompok

Buatlah 6 kelompok dibagi sesuai dengan jumlah mahasiswa:

- Membuat materi presentasi (kreatif, menarik dan mudah dipahami) sesuai dengan judul proyek yang dipilih
- Unggah file PPT pada google drive PJ mata kuliah maksimal pada pertemuan 6 (presentasi dimulai pertemuan 6)
- Setiap mahasiswa menyiapkan minimal 1 pertanyaan untuk kelompok lain

List Proyek

1. Simulasi Penjadwalan CPU (CPU Scheduling Simulator)

Deskripsi:

Mahasiswa membuat program simulasi algoritma penjadwalan proses (FCFS, SJF, Priority, Round Robin).

Tugas:

- Input berupa daftar proses (arrival time, burst time, priority).
- Output berupa Gantt chart, average waiting time, dan turnaround time.
- Bandingkan performa tiap algoritma.

Tujuan: memahami cara kerja scheduler dalam sistem operasi.

List Proyek

2. Manajemen Memori dengan Paging & Page Replacement

Deskripsi:

Mahasiswa membangun simulasi alokasi memori menggunakan paging serta implementasi algoritma penggantian halaman (FIFO, LRU, Optimal).

Tugas:

- Input: sequence akses halaman.
- Output: jumlah *page fault* tiap algoritma.
- Buat analisis perbandingan efisiensi.

Tujuan: memahami manajemen memori dan dampaknya pada performa sistem.

List Proyek

3. Deadlock Detection & Avoidance

Deskripsi:

Proyek membuat simulasi deadlock dengan resource terbatas, lalu menerapkan algoritma Banker untuk pencegahan.

Tugas:

- Simulasikan beberapa proses yang bersaing memperebutkan resource.
- Tunjukkan kondisi deadlock.
- Terapkan Banker's Algorithm untuk menghindarinya.

Tujuan: memahami konsep deadlock, penyebab, serta solusi pencegahan.

List Proyek

4. Sistem File Mini (Mini File System)

Deskripsi:

Kelompok membangun sistem file sederhana dengan fungsi dasar.

Tugas:

- Implementasi fungsi: membuat file, membaca file, menulis isi, menghapus file.
- Struktur data bisa berupa array/linked list.
- Tambahkan fitur sederhana seperti directory.

Tujuan: memahami prinsip sistem file dan pengelolaan data.

List Proyek

5. Multithreading & Sinkronisasi (Producer–Consumer Problem)

Deskripsi:

Proyek membuat simulasi multithreading dengan problem produsen-konsumen menggunakan semaphore atau monitor.

Tugas:

- Buat program dengan buffer terbatas.
- Implementasikan produsen yang menghasilkan data dan konsumen yang mengambil data.
- Pastikan tidak terjadi race condition.

Tujuan: memahami *concurrency* dan sinkronisasi antar proses/thread.

List Proyek

6. Shell Command Interpreter Sederhana

Deskripsi:

Mahasiswa membangun program yang berfungsi sebagai shell sederhana.

Tugas:

- Bisa mengeksekusi perintah dasar OS (misalnya: ls, pwd, mkdir, cat).
- Mendukung input & output redirection (>, <) sederhana.
- (Opsional) dukungan pipe (|).

Tujuan: memahami interaksi antara pengguna, shell, dan kernel.

Deliverables

- **Source code** proyek (20%)
- **Laporan ppt** (latar belakang teori, desain, implementasi, hasil, analisis). (40%)
- **Presentasi.** (40%)

Contoh Proyek: Simulasi Manajemen Antrian I/O dengan Buffering

Deskripsi

Mahasiswa diminta membuat simulasi sistem antrian untuk perangkat I/O (misalnya printer atau disk) dengan menggunakan teknik **buffering**. Program harus dapat menampilkan bagaimana request dari beberapa proses ditangani, baik dengan **single buffering** maupun **double buffering**.

Detail Proses

- **Input:**

- Daftar proses dengan permintaan I/O (misalnya proses A butuh mencetak 5 data, proses B butuh 3 data).
- Kapasitas buffer (misalnya buffer size = 4).

- **Proses yang disimulasikan:**

- Bagaimana request masuk ke buffer.
- Bagaimana sistem melayani request dengan model single buffer dan double buffer.
- Simulasikan antrian jika buffer penuh.

- **Output:**

- Status buffer pada tiap langkah (visualisasi tabel/array).
- Lama waktu proses selesai dengan single buffering vs double buffering.
- Analisis perbandingan kinerja (misalnya waktu tunggu rata-rata).

Tujuan

- Mahasiswa memahami bagaimana sistem operasi mengatur **antrian I/O**.
- Membedakan performa **single buffering** dan **double buffering**.
- Menyadari pentingnya manajemen resource agar sistem lebih efisien.

Deliverables

- **Source code** simulasi (basic).
- **Laporan kelompok** berisi:
 - Latar belakang teori buffering.
 - Desain algoritma simulasi.
 - Hasil simulasi dan analisis performa.
- **Demo presentasi** (menampilkan simulasi berjalan + penjelasan hasil).

Pembagian Tugas

Proyek ini terdiri dari (6 orang):

- 2 orang riset teori buffering & menulis laporan latar belakang.
- 2 orang coding & testing simulasi.
- 2 orang dokumentasi hasil + presentasi.

Pembagian Tugas

Proyek ini terdiri dari (6 orang):

- 2 orang riset teori buffering & menulis laporan latar belakang.
- 2 orang coding & testing simulasi.
- 2 orang dokumentasi hasil + presentasi.

Source Code

```
program IOBufferSimulation;

uses crt, SysUtils;

type
  TProcess = record
    name : string[5];
    data_count : integer;
  end;

var
  processes : array[1..3] of TProcess;
  buffer : array[1..10] of string[20];
  bufferSize : integer;
  i, j, k, timeUnit : integer;
```


Source Code

```
procedure SingleBuffering;
var
  bufCount : integer;
begin
  writeln('=== SIMULASI SINGLE BUFFERING ===');
  bufCount := 0;
  timeUnit := 0;

  for i := 1 to 3 do
    begin
      for j := 1 to processes[i].data_count do
        begin
          inc(timeUnit);
          if bufCount < bufferSize then
            begin
              inc(bufCount);
              buffer[bufCount] := processes[i].name + '-data' + IntToStr(j);
              write('T', timeUnit, ': ', processes[i].name, ' menambahkan data -> [');
            end
          else
            begin
              write('T', timeUnit, ': ', processes[i].name, ' menunggu data [');
            end
          write(']');
        end
      end
    end
  end
```

Source Code

```
for k := 1 to bufCount do
    write(buffer[k], ' ');
    writeln('');
end
else
begin
    writeln('T', timeUnit, ': BUFFER PENUH! ', processes[i].name, ' harus menunggu. ');
    inc(timeUnit);
    writeln('T', timeUnit, ': Buffer dikosongkan oleh device. ');
    bufCount := 1;
    buffer[bufCount] := processes[i].name + '-data' + IntToStr(j);
end;
end;
end;
```

Source Code

```
inc(timeUnit);  
writeln('T', timeUnit, ': Buffer terakhir dikosongkan.');
```

```
writeln('Total waktu single buffering: ', timeUnit, ' unit waktu');  
writeln;  
end;  
  
procedure DoubleBuffering;  
var  
    bufCount : integer;  
    activeBuffer : integer;  
begin  
    writeln('=== SIMULASI DOUBLE BUFFERING ===');  
    bufCount := 0;  
    timeUnit := 0;  
    activeBuffer := 1;
```

Source Code

```
for i := 1 to 3 do
begin
  for j := 1 to processes[i].data_count do
  begin
    inc(timeUnit);
    if bufCount < bufferSize then
    begin
      inc(bufCount);
      buffer[bufCount] := processes[i].name + '-data' + IntToStr(j);
      write('T', timeUnit, ': ', processes[i].name, ' menambahkan data ke buffer aktif -> [');
      for k := 1 to bufCount do
        write(buffer[k], ' ');
      writeln(']');
    end
  end
end
```

Source Code

```
else
  begin
    writeln('T', timeUnit, ': Buffer penuh, switch buffer. ');
    bufCount := 1;
    buffer[bufCount] := processes[i].name + '-data' + IntToStr(j);
  end;
end;
end;

inc(timeUnit);
writeln('T', timeUnit, ': Buffer terakhir dikosongkan. ');
writeln('Total waktu double buffering: ', timeUnit, ' unit waktu');
writeln;
end;
```

Source Code

```
begin
  clrscr;
  bufferSize := 4;

  processes[1].name := 'P1';
  processes[1].data_count := 5;

  processes[2].name := 'P2';
  processes[2].data_count := 3;

  processes[3].name := 'P3';
  processes[3].data_count := 4;

  SingleBuffering;
  DoubleBuffering;
  writeln('=== ANALISIS ===');
  writeln('Bandingkan hasil single buffering dan double buffering.');
```

readln;

```
end.
```

Output Program

```
main.pas
input

=== SIMULASI SINGLE BUFFERING ===
T1: P1 menambahkan data -> [P1-data1 ]
T2: P1 menambahkan data -> [P1-data1 P1-data2 ]
T3: P1 menambahkan data -> [P1-data1 P1-data2 P1-data3 ]
T4: P1 menambahkan data -> [P1-data1 P1-data2 P1-data3 P1-data4 ]
T5: BUFFER PENUH! P1 harus menunggu.
T6: Buffer dikosongkan oleh device.
T7: P2 menambahkan data -> [P1-data5 P2-data1 ]
T8: P2 menambahkan data -> [P1-data5 P2-data1 P2-data2 ]
T9: P2 menambahkan data -> [P1-data5 P2-data1 P2-data2 P2-data3 ]
T10: BUFFER PENUH! P3 harus menunggu.
T11: Buffer dikosongkan oleh device.
T12: P3 menambahkan data -> [P3-data1 P3-data2 ]
T13: P3 menambahkan data -> [P3-data1 P3-data2 P3-data3 ]
T14: P3 menambahkan data -> [P3-data1 P3-data2 P3-data3 P3-data4 ]
T15: Buffer terakhir dikosongkan.
Total waktu single buffering: 15 unit waktu

=== SIMULASI DOUBLE BUFFERING ===
T1: P1 menambahkan data ke buffer aktif -> [P1-data1 ]
T2: P1 menambahkan data ke buffer aktif -> [P1-data1 P1-data2 ]
T3: P1 menambahkan data ke buffer aktif -> [P1-data1 P1-data2 P1-data3 ]
T4: P1 menambahkan data ke buffer aktif -> [P1-data1 P1-data2 P1-data3 P1-data4 ]
T5: Buffer penuh, switch buffer.
T6: P2 menambahkan data ke buffer aktif -> [P1-data5 P2-data1 ]
T7: P2 menambahkan data ke buffer aktif -> [P1-data5 P2-data1 P2-data2 ]
T8: P2 menambahkan data ke buffer aktif -> [P1-data5 P2-data1 P2-data2 P2-data3 ]
T9: Buffer penuh, switch buffer.
T10: P3 menambahkan data ke buffer aktif -> [P3-data1 P3-data2 ]
T11: P3 menambahkan data ke buffer aktif -> [P3-data1 P3-data2 P3-data3 ]
T12: P3 menambahkan data ke buffer aktif -> [P3-data1 P3-data2 P3-data3 P3-data4 ]
T13: Buffer terakhir dikosongkan.
Total waktu double buffering: 13 unit waktu

=== ANALISIS ===
Bandingkan hasil single buffering dan double buffering.
```

Terima kasih